

For and do-while loops

CSC103 Programming Fundamentals / Lecture 11

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Print a 3 by 4 rectangle of # characters using nested loops. Explain how many times the inner body runs.

Repeat three times for the three rows.

Learning objectives

- Select for, while or do-while appropriately
- Explain break and continue
- Trace nested iteration

CLO-2

The for loop

The do-while loop

Break and continue

Nested loops

The for loop

- A for header groups initialisation, condition and update.
- The update runs after each normally completed body.
- A header variable belongs to the loop scope.

Example

```
for (int i = 0; i < 4; i++) {  
    System.out.println(i);  
}
```

The do-while loop

- do-while checks its condition after the body.
- The body therefore runs at least once.
- The statement ends with a semicolon.

Example

```
int n = 0;
do {
    System.out.println(n);
    n++;
} while (n < 0);
```

Break and continue

- break exits the nearest loop.
- continue skips the rest of the current iteration.
- In a for loop, continue proceeds to the update step.

Example

```
for (int i = 1; i <= 5; i++) {  
    if (i == 2) continue;  
    if (i == 4) break;  
    System.out.println(i);  
}
```

Nested loops

- The inner loop completes for each outer iteration.
- With independent bounds, body executions multiply.
- Choose distinct names for separate counters.

Example

```
for (int row = 1; row <= 2; row++) {  
    for (int col = 1; col <= 3; col++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

The for update follows continue

- A for loop transfers from continue to its update expression.
- A while loop transfers from continue directly to its condition.
- This difference matters when progress depends on a body statement that continue might skip.

Example

```
for (...; ...; i++) {  
    if (skip) continue;  
}  
The i++ update still runs.
```


Row boundaries belong to the outer loop

- The inner loop prints all characters in one row.
- A newline after the inner loop ends that row.
- Putting the newline inside the inner loop would produce one character per line.

Example

```
Row 1: ##### then newline  
Row 2: ##### then newline  
Row 3: ##### then newline
```

Nested loops visit a rectangular grid

	column 0	column 1	column 2
row 0	r0c0	r0c1	r0c2
row 1	r1c0	r1c1	r1c2

What to notice

The inner loop completes a row before the outer loop moves to the next row.

Key distinctions

Loop	Condition check	Minimum body executions
while	Before body	0
for	Before body	0
do-while	After body	1

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    sum += i;
}
System.out.println(sum);
```

First example: result

12

The added values are 1, 2, 4 and 5.
continue still allows the for update to run.

Break and continue
Nested loops

Guided practice

Print a 3 by 4 rectangle of # characters using nested loops. Explain how many times the inner body runs.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Repeat three times for the three rows.
- Inside each row, print four # characters without ending the line.
- Print one newline after completing the inner loop.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution11 {  
    public static void main(String[] args) throws Exception {  
        for (int row = 0; row < 3; row++) {  
            for (int column = 0; column < 4; column++) {  
                System.out.print("#");  
            }  
            System.out.println();  
        }  
    }  
}
```


Execution trace

Outer row	Inner column values	Printed content	Inner executions
0	0, 1, 2, 3	####	4
1	0, 1, 2, 3	####	4
2	0, 1, 2, 3	####	4
All rows	3 rows x 4 columns	Three lines	12

Follow the changing state in execution order.

Solution result

```
####  
####  
####
```

Why this result follows
Print one newline after completing
the inner loop.

A common incorrect approach

```
int i=0;
while(i<5) {
  if(i==2) continue;
  i++;
}
```

Example

At $i=2$, `continue` skips the update forever. Move the update before any `continue` path or use a `for` loop.

Boundary and failure checks

Check the stated input assumptions.

Print a multiplication table for 1 through 5. Use formatted output and nested loops.

Outer loop: three iterations. Inner loop: four iterations per row. Print a newline after the inner loop. Total inner body executions: 12.

Check your understanding

Which loop guarantees one body execution? Does break exit every nested loop?

Write a prediction and explain the rule that supports it.

Answer and explanation

do-while. An unlabelled break exits only the nearest enclosing loop or switch.

At $i=2$, continue skips the update forever. Move the update before any continue path or use a for loop.

Compare cases and predict outcomes

Outer index	Inner indexes	Body executions
0	0, 1, 2	3
1	0, 1, 2	3
Total	2 rows x 3 columns	6

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Print two rows of three stars. Predict what changes if `println()` is moved inside the inner loop.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
for (int row = 0; row < 2; row++) {  
    for (int col = 0; col < 3; col++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

- The intended output has two lines, each containing ***.
- The newline belongs after the inner loop.
- Moving it inside prints one star per line, producing six lines.

A multiplication table with nested loops

- The row counter supplies one factor and the column counter supplies the other.
- Reset the column loop for each new row.
- Formatting belongs inside the inner loop; the newline belongs after it.

Source: [supplied ISB campus slides](#). See [speaker notes](#) and [ISB_Source_Map.md](#).

Worked problem: A multiplication table with nested loops

Task

Print the 1-through-3 multiplication table using nested for loops.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture11 {  
    public static void main(String[] args) throws Exception {  
        for (int row = 1; row <= 3; row++) {  
            for (int col = 1; col <= 3; col++) {  
                System.out.printf("%3d", row * col);  
            }  
            System.out.println();  
        }  
    }  
}
```

Worked trace and expected result

Row factor	Column factors	Printed products
1	1, 2, 3	1, 2, 3
2	1, 2, 3	2, 4, 6
3	1, 2, 3	3, 6, 9

Expected console output

```
1 2 3
2 4 6
3 6 9
```

Extend the ISB example

Practice

How many inner-body executions and newline calls occur for a 10-by-10 table?

Explain your reasoning

100 product-printing executions and 10 newline calls. The two loop bounds multiply.

Lecture summary

- for, while or do-while appropriately
 - break and continue
 - nested iteration
- Repeat three times for the three rows.
 - Inside each row, print four # characters without ending the line.
 - Print one newline after completing the inner loop.

After-class practice

Print a multiplication table for 1 through 5. Use formatted output and nested loops.

Syllabus reading

Deitel Ch 8 (as listed)

Next lecture: References and string input